

1 Basics

```

1  #include <bits/stdc++.h>
2  #define MIN_IN(A) *(std::min_element(A.begin(), A.end()))
3  #define MAX_IN(A) *(std::max_element(A.begin(), A.end()))
4  #define MAX_AT(A) (std::max_element(A.begin(), A.end()) - A.begin())
5  #define MIN_AT(A) (std::min_element(A.begin(), A.end()) - A.begin())
6  #define COUNT(V,A) count(V.begin(), V.end(), A);
7  #define GCD(a,b) std::__gcd(a,b);
8  #define LCM(a, b) ((a) / std::__gcd((a), (b)) * (b))
9  #define SORT(A) std::sort(A.begin(), A.end());
10 #define RSORT(A) std::sort(A.rbegin(), A.rend());
11 #define REVERSE(A) std::reverse(A.begin(), A.end());
12 accumulate(v.begin(), v.end(), 0LL);
13 sort(v.begin(), v.end(), comp);           // custom comparator
14
15 __builtin_popcount(int number);
16 __builtin_popcountll(long long number);
17 __builtin_clz(x);           // leading zeros in unsigned int
18 __builtin_ctz(x);           // trailing zeros
19 toupper(); tolower(); // Character case change
20
21 binary_search(v.begin(), v.end(), x); // true/false
22 lower_bound(v.begin(), v.end(), x);   // first ≥ x
23 upper_bound(v.begin(), v.end(), x);   // first > x
24 int idx = lower_bound(v.begin(), v.end(), x) - v.begin(); // index trick
25
26 next_permutation(v.begin(), v.end());
27 prev_permutation(v.begin(), v.end());

```

2 STL functions

```

1  ios::sync_with_stdio(false);
2  cin.tie(nullptr);
3
4  set<int> s;
5  s.insert(x);
6  s.erase(x);
7  s.count(x);
8  s.lower_bound(x);
9
10 multiset<int> ms;
11 ms.insert(x);
12 ms.erase(ms.find(x)); // erase single occurrence
13
14 map<int, int> mp;
15 mp[x]++;
16 mp.erase(x);
17 mp.count(x);
18
19 priority_queue<int> pq;
20 priority_queue<int, vector<int>, greater<int>> pq;
21
22 s.length()
23 s.substr(l, len)
24 s.find("abc")
25 stoi(s)
26 to_string(x)
27
28 INT_MAX
29 INT_MIN
30 LLONG_MAX
31
32 // Use this combination to delete all duplicates in vector.
33 sort(v.begin(), v.end());
34 v.erase(unique(v.begin(), v.end()), v.end());

```

3 Bitset

```
1 #include <bitset>
2
3 bitset<8> b;           // 00000000
4 bitset<8> b1(5);       // 00000101
5 bitset<8> b2("10110010"); // string constructor
6
7 b[i]                  // read bit i (0-indexed, from right)
8 b[i] = 1;
9 b.set(i);             // set bit i to 1
10 b.reset(i);           // set bit i to 0
11 b.flip(i);            // toggle bit i
12
13 b.set();              // all 1s
14 b.reset();            // all 0s
15 b.flip();             // toggle all bits
16
17 b.count();            // number of set bits
18 b.any();              // at least one bit = 1?
19 b.none();             // all bits = 0?
20 b.all();              // all bits = 1?
21 b.size();             // total number of bits
22
23 b.to_string();        // "01010101"
24 b.to_ulong();         // unsigned long (≤ 64 bits)
25 b.to_ullong();        // unsigned long long
```

4 Custom comparator

```
1 bool customComparison(int a, int b)
2 {
3     // Custom comparison logic
4     return a < b; // it sorts in ascending order
5 }
6 sort(myVec.begin(), myVec.end(), customComparison);
7
8 // Method 2
9 sort(myVec.begin(), myVec.end(), [](int a, int b) {
10     return a < b; // this sorts in ascending order
11 });
```

5 Modular inverse

```
1 // Method 1: When mod is prime
2 long long modexp(long long a, long long e, long long mod){
3     long long res = 1;
4     a %= mod;
5     while(e){
6         if(e & 1) res = res * a % mod;
7         a = a * a % mod;
8         e >>= 1;
9     }
10    return res;
11 }
12
13 long long modinv(long long a, long long mod){
14     return modexp(a, mod - 2, mod);
15 }
16
17 // Method 2: gcd(a,mod) = 1 is necessary condition for mod inv to exist.
18 long long extended_gcd(long long a, long long b, long long &x, long long &y){
19     if(b == 0){
20         x = 1; y = 0;
21         return a;
22     }
23     long long x1, y1;
24     long long g = extended_gcd(b, a % b, x1, y1);
25     x = y1;
```

```

26     y = x1 - y1 * (a / b);
27     return g;
28 }
29
30 long long modinv(long long a, long long mod){
31     long long x, y;
32     long long g = extended_gcd(a, mod, x, y);
33     if(g != 1) return -1; // inverse doesn't exist
34     return (x % mod + mod) % mod;
35 }
36
37 // Method 3: Precompute modinv for many quantities when mod is prime
38 vector<long long> inv(n+1);
39 inv[1] = 1;
40 for(int i = 2; i ≤ n; i++)
41     inv[i] = mod - (mod/i) * inv[mod%i] % mod;
42
43 // Also calculating nCr (when mod is prime)
44 const int MOD = 1000000007;
45 const int MAXN = 200000; // set as needed
46 long long fact[MAXN+1], invfact[MAXN+1];
47
48 void init(){
49     fact[0] = 1;
50     for(int i = 1; i ≤ MAXN; i++)
51         fact[i] = fact[i-1] * i % MOD;
52
53     invfact[MAXN] = modexp(fact[MAXN], MOD - 2);
54     for(int i = MAXN; i > 0; i--)
55         invfact[i-1] = invfact[i] * i % MOD;
56 }
57
58 long long nCr(int n, int r){
59     if(r < 0 || r > n) return 0;
60     return fact[n] * invfact[r] % MOD * invfact[n-r] % MOD;
61 }

```

6 Minimum segment Tree

```

1  vector<int> t(4*n+1,0);
2
3  // v=1, tl=0, tr=n-1 always for calling below functions. 0-indexed parameters.
4
5  void buildmin(vector<int>&a, int v, int tl, int tr, vector<int> &t) {
6      if (tl == tr) {
7          t[v] = a[tl];
8      } else {
9          int tm = (tl + tr) / 2;
10         buildmin(a, v*2, tl, tm, t);
11         buildmin(a, v*2+1, tm+1, tr, t);
12         t[v] = min(t[v*2], t[v*2+1]);
13     }
14 }
15
16 int mini(int v, int tl, int tr, int l, int r, vector<int> &t) {
17     if (l > r)
18         return INT_MAX;
19     if (l == tl && r == tr) {
20         return t[v];
21     }
22     int tm = (tl + tr) / 2;
23     return min(mini(v*2, tl, tm, l, min(r, tm), t),
24               ,mini(v*2+1, tm+1, tr, max(l, tm+1), r, t));
25 }
26
27 void updatemin(int v, int tl, int tr, int pos, int new_val, vector<int> &t) {
28     if (tl == tr) {
29         t[v] = new_val;
30     } else {
31         int tm = (tl + tr) / 2;
32         if (pos ≤ tm)
33             updatemin(v*2, tl, tm, pos, new_val, t);
34         else
35             updatemin(v*2+1, tm+1, tr, pos, new_val, t);
36         t[v] = min(t[v*2], t[v*2+1]);
37     }

```

7 Disjoint Set Union

```

1     struct DSU {
2     vector<int> par, rnk, sz;
3     int c;
4     DSU(int n) : par(n + 1), rnk(n + 1, 0), sz(n + 1, 1), c(n) {
5         for (int i = 1; i ≤ n; ++i) par[i] = i;
6     }
7     int find(int i) {
8         return (par[i] == i ? i : (par[i] = find(par[i])));
9     }
10    bool same(int i, int j) {
11        return find(i) == find(j);
12    }
13    int get_size(int i) {
14        return sz[find(i)];
15    }
16    int count() {
17        return c;    //connected components
18    }
19    int merge(int i, int j) {
20        if ((i = find(i)) == (j = find(j))) return -1;
21        else —c;
22        if (rnk[i] > rnk[j]) swap(i, j);
23        par[i] = j;
24        sz[j] += sz[i];
25        if (rnk[i] == rnk[j]) rnk[j]++;
26        return j;
27    }
28 };

```

8 Policy-based Data Structures

```

1  #include<ext/pb_ds/assoc_container.hpp>
2  #include<ext/pb_ds/tree_policy.hpp>
3  using namespace __gnu_pbds;
4
5  template <typename T> using o_set = tree<T, null_type, less<T>, rb_tree_tag, ...
6      tree_order_statistics_node_update>;
7
8  template <typename T, typename R> using o_map = tree<T, R, less<T>, rb_tree_tag, ...
9      tree_order_statistics_node_update>;
10
11 o_set<int> se;
12 cout << *se.find_by_order(0) << endl;    ///k th element
13 cout << se.order_of_key(2) << endl;    ///number of elements less than k
14
15 o_map<int, int> mp;
16 cout << mp.find_by_order(0)→second << endl;    ///k th element
17 cout << mp.order_of_key(2) << endl;    ///number of first elements less than k

```

9 Binary Lifting

```

1  int timer;
2  vector<int> tin, tout;
3  vector<vector<int>>> up;
4
5  void dfs(int v, int p)
6  {
7      tin[v] = ++timer;
8      up[v][0] = p;
9      for (int i = 1; i ≤ l; ++i)
10         up[v][i] = up[up[v][i-1]][i-1];
11
12     for (int u : adj[v]) {
13         if (u != p)
14             dfs(u, v);
15     }
16 }

```

```

15     }
16
17     tout[v] = ++timer;
18 }
19
20 bool is_ancestor(int u, int v)
21 {
22     return tin[u] ≤ tin[v] && tout[u] ≥ tout[v];
23 }
24
25 int lca(int u, int v)
26 {
27     if (is_ancestor(u, v))
28         return u;
29     if (is_ancestor(v, u))
30         return v;
31     for (int i = 1; i ≥ 0; —i) {
32         if (!is_ancestor(up[u][i], v))
33             u = up[u][i];
34     }
35     return up[u][0];
36 }
37
38 void preprocess(int root) {
39     tin.resize(n);
40     tout.resize(n);
41     timer = 0;
42     l = ceil(log2(n));
43     up.assign(n, vector<int>(l + 1));
44     dfs(root, root);
45 }

```

10 Strongly connected components (Kosaraju)

```

1  vector<bool> visited; // keeps track of which vertices are already visited
2
3  // runs depth first search starting at vertex v.
4  // each visited vertex is appended to the output vector when dfs leaves it.
5  void dfs(int v, vector<vector<int>> const& adj, vector<int> &output) {
6      visited[v] = true;
7      for (auto u : adj[v])
8          if (!visited[u])
9              dfs(u, adj, output);
10     output.push_back(v);
11 }
12
13 // input: adj — adjacency list of G
14 // output: components — the strongly connected components in G
15 // output: adj_cond — adjacency list of G^TSCC (by root vertices)
16 void strongly_connected_components(vector<vector<int>> const& adj,
17                                   vector<vector<int>> &components,
18                                   vector<vector<int>> &adj_cond) {
19     int n = adj.size();
20     components.clear(), adj_cond.clear();
21
22     vector<int> order; // will be a sorted list of G's vertices by exit time
23
24     visited.assign(n, false);
25
26     // first series of depth first searches
27     for (int i = 0; i < n; i++)
28         if (!visited[i])
29             dfs(i, adj, order);
30
31     // create adjacency list of G^T
32     vector<vector<int>> adj_rev(n);
33     for (int v = 0; v < n; v++)
34         for (int u : adj[v])
35             adj_rev[u].push_back(v);
36
37     visited.assign(n, false);
38     reverse(order.begin(), order.end());
39
40     vector<int> roots(n, 0); // gives the root vertex of a vertex's SCC
41
42     // second series of depth first searches

```

```

43     for (auto v : order)
44         if (!visited[v]) {
45             std::vector<int> component;
46             dfs(v, adj_rev, component);
47             components.push_back(component);
48             int root = *component.begin();
49             for (auto u : component)
50                 roots[u] = root;
51         }
52
53     // add edges to condensation graph
54     adj_cond.assign(n, {});
55     for (int v = 0; v < n; v++)
56         for (auto u : adj[v])
57             if (roots[v] != roots[u])
58                 adj_cond[roots[v]].push_back(roots[u]);
59 }

```

11 Minimum spanning Tree (Kruskal's) ($O(M \log N)$)

```

1  vector<int> parent, rank;
2
3  void make_set(int v) {
4      parent[v] = v;
5      rank[v] = 0;
6  }
7
8  int find_set(int v) {
9      if (v == parent[v])
10         return v;
11     return parent[v] = find_set(parent[v]);
12 }
13
14 void union_sets(int a, int b) {
15     a = find_set(a);
16     b = find_set(b);
17     if (a != b) {
18         if (rank[a] < rank[b])
19             swap(a, b);
20         parent[b] = a;
21         if (rank[a] == rank[b])
22             rank[a]++;
23     }
24 }
25
26 struct Edge {
27     int u, v, weight;
28     bool operator<(Edge const& other) {
29         return weight < other.weight;
30     }
31 };
32
33 int n;
34 vector<Edge> edges;
35
36 int cost = 0;
37 vector<Edge> result;
38 parent.resize(n);
39 rank.resize(n);
40 for (int i = 0; i < n; i++)
41     make_set(i);
42
43 sort(edges.begin(), edges.end());
44
45 for (Edge e : edges) {
46     if (find_set(e.u) != find_set(e.v)) {
47         cost += e.weight;
48         result.push_back(e);
49         union_sets(e.u, e.v);
50     }
51 }

```

12 Floyd-Warshall Algorithm ($O(n^3)$)

```
1   for (int k = 0; k < n; ++k) {
2   for (int i = 0; i < n; ++i) {
3       for (int j = 0; j < n; ++j) {
4           if (d[i][k] < INF && d[k][j] < INF)
5               d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
6       }
7   }
8 }
```

13 SPFA ($O(nm)$)

```
1  const int INF = 1000000000;
2  vector<vector<pair<int, int>>> adj;
3
4  bool spfa(int s, vector<int>& d) {
5      int n = adj.size();
6      d.assign(n, INF);
7      vector<int> cnt(n, 0);
8      vector<bool> inqueue(n, false);
9      queue<int> q;
10
11     d[s] = 0;
12     q.push(s);
13     inqueue[s] = true;
14     while (!q.empty()) {
15         int v = q.front();
16         q.pop();
17         inqueue[v] = false;
18
19         for (auto edge : adj[v]) {
20             int to = edge.first;
21             int len = edge.second;
22
23             if (d[v] + len < d[to]) {
24                 d[to] = d[v] + len;
25                 if (!inqueue[to]) {
26                     q.push(to);
27                     inqueue[to] = true;
28                     cnt[to]++;
29                     if (cnt[to] > n)
30                         return false; // negative cycle
31                 }
32             }
33         }
34     }
35     return true;
36 }
```

14 Sieve of eratosthenes

```
1  template <typename T>
2  void sieve(T n, vector<T> &v){
3      vector<bool> isnotprime(n+1, false);
4      isnotprime[0] = true;
5      isnotprime[1] = true;
6      vector<T> primes;
7      for (T i=2; i*i<=n; i++){
8          if (!isnotprime[i]){
9              for (T j=i*i; j<=n; j+=i){
10                 isnotprime[j] = true;
11             }
12         }
13     }
14     for (T i=2; i<=n; i++){
15         if (!isnotprime[i]){
16             v.push_back(i);
17         }
18     }
19 }
```

```

18     }
19 }
20 return;
21 }

```

15 Matrix exponentiation

```

1  using ll = long long;
2  const ll MOD = 1e9 + 7;
3
4  vector<vector<ll>> multiply(
5      const vector<vector<ll>> &A,
6      const vector<vector<ll>> &B
7  ) {
8      int n = A.size();
9      vector<vector<ll>> C(n, vector<ll>(n, 0));
10
11     for (int i = 0; i < n; i++) {
12         for (int k = 0; k < n; k++) {
13             if (A[i][k] == 0) continue;
14             for (int j = 0; j < n; j++) {
15                 C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
16             }
17         }
18     }
19     return C;
20 }
21
22 vector<vector<ll>> matrix_power(
23     vector<vector<ll>> A,
24     long long power
25 ) {
26     int n = A.size();
27     vector<vector<ll>> result = identity(n);
28
29     while (power > 0) {
30         if (power & 1)
31             result = multiply(result, A);
32         A = multiply(A, A);
33         power >>= 1;
34     }
35     return result;
36 }

```

16 Convex Hull

```

1  struct pt {
2      double x, y;
3      bool operator == (pt const& t) const {
4          return x == t.x && y == t.y;
5      }
6  };
7
8  int orientation(pt a, pt b, pt c) {
9      double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
10     if (v < 0) return -1; // clockwise
11     if (v > 0) return +1; // counter-clockwise
12     return 0;
13 }
14
15 bool cw(pt a, pt b, pt c, bool include_collinear) {
16     int o = orientation(a, b, c);
17     return o < 0 || (include_collinear && o == 0);
18 }
19 bool collinear(pt a, pt b, pt c) { return orientation(a, b, c) == 0; }
20
21 void convex_hull(vector<pt>& a, bool include_collinear = false) {
22     pt p0 = *min_element(a.begin(), a.end(), [](pt a, pt b) {
23         return make_pair(a.y, a.x) < make_pair(b.y, b.x);
24     });
25     sort(a.begin(), a.end(), [&p0](const pt& a, const pt& b) {

```



```

26     int o = orientation(p0, a, b);
27     if (o == 0)
28         return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
29             < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
30     return o < 0;
31 });
32 if (include_collinear) {
33     int i = (int)a.size()-1;
34     while (i ≥ 0 && collinear(p0, a[i], a.back())) i--;
35     reverse(a.begin()+i+1, a.end());
36 }
37
38 vector<pt> st;
39 for (int i = 0; i < (int)a.size(); i++) {
40     while (st.size() > 1 && !cw(st[st.size()-2], st.back(), a[i], include_collinear))
41         st.pop_back();
42     st.push_back(a[i]);
43 }
44
45 if (include_collinear == false && st.size() == 2 && st[0] == st[1])
46     st.pop_back();
47
48 a = st;
49 }

```

17 Articulation Points ($O(n+m)$)

```

1  int n; // number of nodes
2  vector<vector<int>>> adj; // adjacency list of graph
3
4  vector<bool> visited;
5  vector<int> tin, low;
6  int timer;
7
8  void dfs(int v, int p = -1) {
9      visited[v] = true;
10     tin[v] = low[v] = timer++;
11     int children=0;
12     for (int to : adj[v]) {
13         if (to == p) continue;
14         if (visited[to]) {
15             low[v] = min(low[v], tin[to]);
16         } else {
17             dfs(to, v);
18             low[v] = min(low[v], low[to]);
19             if (low[to] ≥ tin[v] && p!= -1)
20                 IS_CUTPOINT(v);
21             ++children;
22         }
23     }
24     if (p == -1 && children > 1)
25         IS_CUTPOINT(v);
26 }
27
28 void find_cutpoints() {
29     timer = 0;
30     visited.assign(n, false);
31     tin.assign(n, -1);
32     low.assign(n, -1);
33     for (int i = 0; i < n; ++i) {
34         if (!visited[i])
35             dfs(i);
36     }
37 }

```

18 Headers

```

1  #include <iostream> // cin, cout, cerr
2  #include <vector> // vector
3  #include <array> // array
4  #include <deque> // deque

```

```

5 #include <list>           // list
6 #include <forward_list>   // forward_list
7 #include <stack>          // stack
8 #include <queue>          // queue, priority_queue
9 #include <set>             // set, multiset
10 #include <map>            // map, multimap
11 #include <unordered_set>  // unordered_set, unordered_multiset
12 #include <unordered_map>  // unordered_map, unordered_multimap
13 #include <string>         // string, stoi, to_string
14 #include <cstring>        // memset, strcpy (C-style strings)
15 #include <sstream>        // stringstream
16 #include <algorithm>      // sort, min, max, reverse, lower_bound, upper_bound
17 #include <numeric>        // accumulate, gcd, lcm
18 #include <functional>     // greater, less, function
19 #include <utility>        // pair, swap, move
20 #include <tuple>          // tuple, tie
21 #include <cmath>          // sqrt, pow, abs, sin, cos, etc.
22 #include <bitset>         // bitset

```

19 Chinese Remainder Theorem

```

1 int gcd(int a, int b, ll& x, ll& y) {
2     x = 1, y = 0;
3     ll x1 = 0, y1 = 1;
4     int a1 = a, b1 = b;
5     while (b1) {
6         int q = a1 / b1;
7         tie(x, x1) = make_tuple(x1, x - static_cast<long long>(q) * x1);
8         tie(y, y1) = make_tuple(y1, y - static_cast<long long>(q) * y1);
9         tie(a1, b1) = make_tuple(b1, a1 - q * b1);
10    }
11    return a1;
12 }
13 pair<int, int> strong_crt(vector<int> &A, vector<int> &M){
14     if(A.size() != M.size()) return {-1, -1};
15     int n = A.size();
16     int a1 = A[0];
17     int m1=M[0];
18
19     for (int i=1; i<n; i++){
20         int a2 = A[i];
21         int m2 = M[i];
22         ll x, y;
23         int g = gcd(m1, m2, x, y);
24         if ( (a1-a2)%g ) return {-1, -1};
25         ll p, q;
26         gcd(m1/g, m2/g, p, q);
27         int mod = m1 / g * m2;
28         int x = static_cast<int>((a1*(m2/g)*q + a2*(m1/g)*p)) % mod;
29         if (a1 < 0) a1 += mod; /** Result is not suppose to be negative*/
30         m1 = mod;
31     }
32     if(a1<0) a1+=m1;
33     return {a1, m1};
34 }

```

ALL THE BEST!